g2(τ) Emitter Classification Pipeline

Code accompanying the paper "*Physics-Based versus Data-Driven Classification of Single-Photon Quantum Emitters from Sparse Autocorrelation Data*." Benchmarks three classification approaches—Levenberg–Marquardt fitting, sequential Bayesian inference, and a feedforward neural network—for distinguishing single-photon emitters (N=1) from multi-emitter sources (N≥2) from Hanbury Brown–Twiss $g^2(\tau)$ autocorrelation data, with particular attention to performance under sparse (short-integration-time) conditions.

What's in this repository

Only the **real experimental data** is included here (together with the small lookup table iter_to_time_mapping_lookup.csv). Simulated training/test data is not, for space reasons—anyone wishing to reproduce or extend the analysis needs to generate it themselves from the provided real data, using the three Step_ scripts below. This is by design: it also means anyone re-running the pipeline is using the exact same generation code we did, not a static snapshot of one particular run.

├── Experimental_Data_Raw/ <- provided: 25 real HBT files

├── Step_00_Correct_and_relabel_experimental_data.py

├── Step_01_Extract_Params_from_experimental_data.py

├── Step_02_Generate_g2_traces_with_exp_params.py

├── LM_analysis_Simulation_data_iterspecific_smoothed.py

├── Bayesian_analysis_simulation_data_iterspecific.py

├── Encoder_analysis_simulation_data_smoothed.py

├── run_seed_sweep.py

├── summarize_seed_sweep.py

├── summarize_seed_convergence_overlap.py

├── summarize_time_to_operating_point.py

├── summarize_method_selection.py

├── make_table1_barplot.py

├── Compute_iter_to_integration_time_mapping.py

├── iter_to_time_mapping_lookup.csv <- provided

├── Figure_real_vs_simulated_with_mapping.py

├── LM_analysis_fixed_tau1.py

├── Benchmark_computational_cost.py

├── fit_pedestal_A2.py

├── Figure_S_emitter_count_rates.py

├── Documentation_and_Instructions.pdf

└── README.md

Running the pipeline (see below) will additionally create:

├── Experimental_Data/ <- created by Step_00

├── experimental_calibration.csv, calibration_summary.csv <- created by Step_01

├── Simulation_Results/ <- created by Step_02, read by all three algorithms

├── seed_sweep_results/ <- created by run_seed_sweep.py (optional split-stability analysis)

├── iter_to_time_mapping_detailed.csv <- optional (supporting analyses)

├── iter_to_time_mapping.png

├── figure_real_vs_simulated_with_mapping.png

├── LM_fixed_tau1_*.csv

├── Benchmark_computational_cost.csv

├── Benchmark_computational_cost_environment.txt

├── A2_fits.csv

├── Emitter_count_statistics.csv

├── Emitter_count_statistics_summary.csv

├── Figure_S_emitter_count_rates.png

Requirements

Python 3.9+ with:

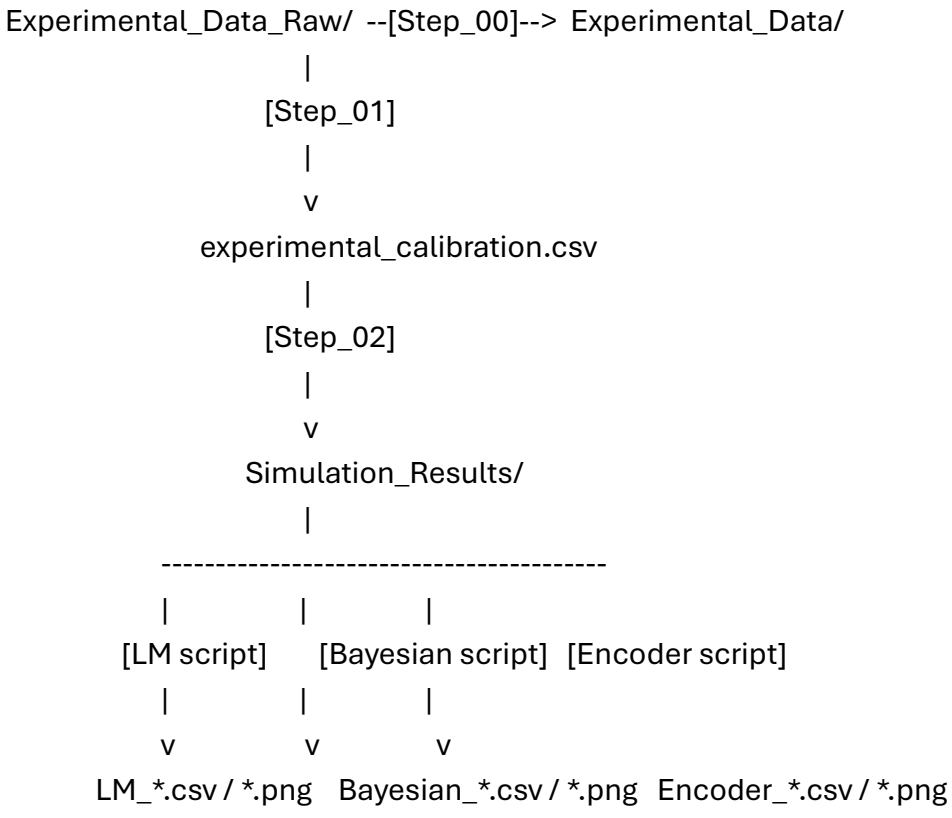
numpy

pandas
scipy
matplotlib
seaborn
scikit-learn

No GPU or deep learning framework is required—the neural-network classifier uses scikit-learn's MLPClassifier, not PyTorch/TensorFlow.

Pipeline overview

The six scripts run in a strict sequence: each one reads the previous step's output. There is no orchestration script — run them one at a time from the command line (python Step_00_...py, etc.), in the same working directory, in this order:



Experimental_Data_Raw/ — real data format (provided)

Each file is one HBT measurement, named <anything>rep_<ID>_n<N>.csv or <anything>rep_<ID>_n<N>_raw.csv (the _raw suffix and any prefix before rep_ are both optional — Step_00 matches on the rep_<ID>_n<N> part). <N> is the emitter number as originally labeled during acquisition; Step_00 re-derives and may correct this (see below).

File structure (CSV, no index column):

- **Row 1**: APD1, followed by one count-rate value (Hz) per iteration column — the single-channel count rate on detector 1 at each integration time.
- **Row 2**: APD2, same for detector 2.
- **Row 3** (header row): tau, iter_1, iter_2, ..., iter_9.
- **Remaining rows**: tau (ns) plus one column of **raw, unnormalized** coincidence counts per iteration — each iter_k column is the *cumulative* histogram after that many integration steps (i.e. iter_9 contains all counts accumulated up to and including the longest integration time, not just that step's counts in isolation).

The integration time (seconds) corresponding to each iter_k column is fixed by acquisition schedule, not stored in the file itself — it's hardcoded in Step_00 and Step_01 as [1, 2, 5, 10, 20, 50, 100, 200, 500] for iter_1...iter_9. **If your acquisition used a different schedule, update ITERATION_INTEGRATION_TIMES_S in both Step_00 and Step_01 before running anything.**

Optionally, a file ending in _norm.csv (otherwise identically named) can accompany a _raw.csv file — the instrument's own normalized $g^2(\tau)$ output for the same measurement. If present, Step_00 uses it to derive an exact baseline; if absent, Step_00 falls back to computing the baseline from the APD rates directly (accurate to within 1-3%, validated against files where both are available).

The folder also contains two plotting helpers, Plot_g2_at_iteration.py and Plot_g2_at_iteration_column.py, which show the $g^2(\tau)$ of one real file at selected integration times (a multi-panel and a single-column layout; each panel is normalized with the accidental-coincidence baseline and corrected for the timing offset). They write g2_at_iterations.png and g2_at_iterations_column.png (the integration times to show are set in SETTINGS) and are not part of the pipeline.

Step_00 — Step_00_Correct_and_relabel_experimental_data.py
Reads: Experimental_Data_Raw/ **Writes**: Experimental_Data/
Corrects a small, fixed timing offset between the two detector channels (typically a few nanoseconds to tens of nanoseconds, detected per file and re-centered so the antibunching dip sits at $\tau=0$), and independently re-derives each file's true emitter number from a classical amplitude threshold on the corrected $g^2(0)$ — since a file's original N label (from the filename) may not survive timing correction. Files whose corrected $g^2(0)$ falls too close to a

classification threshold are excluded as genuinely ambiguous rather than forced into a class.

For each surviving file, writes two outputs to Experimental_Data/:

- rep_<ID>_n<N>.csv — same tau/iter_ column format as the input, but corrected and properly normalized to $g^2(\tau)$.
- rep_<ID>_n<N>_rawcounts.csv — a companion file with the same corrected data as raw (unnormalized) counts, used downstream for proper Poisson-weighted fitting.

Prints a full audit log (detected shift, both $g^2(0)$ estimation methods, and the reason for every exclusion) to the console — worth reading through once to sanity-check your own data before proceeding.

Run: python Step_00_Correct_and_relabel_experimental_data.py

Step_01 — Step_01_Extract_Params_from_experimental_data.py
Reads: Experimental_Data/ (Step_00's output) **Writes**: experimental_calibration.csv, calibration_summary.csv
Extracts, from each corrected real file, the background fraction f_bg (and bunching pedestal A2) needed to calibrate realistic noise levels in the synthetic data generated by Step_02. f_bg = B/(S+B), recovered by inverting the emitter-dilution relation $g^2(0) = 1 - \rho^2/N$ for the file's known N and measured $g^2(0)$ — not measured directly.

Results are pooled by emitter number: N=1 on its own, and N=2 through 5 pooled together (individually there are usually too few real multi-emitter files to characterize each N separately). If even the pooled N≥2 group is too small to be confident, N=1 data is blended in as a fallback — this and every other pooling decision is printed to the console.

Run: python Step_01_Extract_Params_from_experimental_data.py

Step_02 — Step_02_Generate_g2_traces_with_exp_params.py
Reads: experimental_calibration.csv (Step_01's output) **Writes**: Simulation_Results/
Generates the large synthetic dataset the three classifiers are actually trained and evaluated on — a Gillespie Monte Carlo simulation of a 3-level emitter, with the background fraction for each simulated file bootstrapped from the real calibration pool (rather than an idealized constant), so simulated noise statistics match what was actually measured.

Key parameters, all set near the top of the if __name__ == "__main__":-equivalent section of the script (search for TWEAKABLE GENERATION PARAMETERS):

Parameter	Meaning
repetitions	Number of simulated files to generate (5000 in the paper)
max_emitters	Emitter number is drawn uniformly from 1 to this value (5 in the paper — gives ~20% Single / ~80% Multi)
iterations	Number of accumulated integration-time snapshots per file (300 in the paper)
F_BG_OVERRIDE	Set to a number instead of None to force every file to the same background fraction, bypassing the real-data bootstrap (useful for controlled experiments)

This step is the most expensive one to run — expect a runtime of several hours for 5000 repetitions at 300 iterations each, depending on hardware. Consider reducing repetitions for a first test run.

Outputs, one pair per repetition: g2_raw_rep_<ID>_n<N>.csv (thinned raw counts) and g2_norm_rep_<ID>_n<N>.csv (normalized $g^2(\tau)$), plus a single training_labels.csv recording every repetition's ground-truth emitter number, injected background fraction, and a self-consistency check (background fraction and $g^2(0)$ re-extracted from the simulation's own output, for comparison against what was injected).

Run: python Step_02_Generate_g2_traces_with_exp_params.py

The three classifiers

All three read from Simulation_Results/ (Step_02's output) and share:

- The same binary task: N=1 ("Single") vs. N≥2 ("Multi").
- The same 5-fold stratified cross-validation (n_folds=5, random_state=42) — every file is used as a held-out test file exactly once, by a model/threshold that never saw it during calibration or training.
- The same set of 30 evaluated integration-time points (iteration_stride=10, i.e. iterations 10, 20, ..., up to iterations set in Step_02).

- The same three output types: a metrics-vs-iteration table/plot (Recall, Precision, Leakage — see the paper's Methods section for definitions), a confusion matrix at the final iteration, and a per-file correctness heatmap across all iterations — so all three methods' outputs are directly comparable.

No settings need to change between scripts for a first run; each script's SETTINGS dictionary near the top can be edited if you want to point at a different Simulation_Results folder, change the number of folds, or adjust the smoothing strength (ema_alpha, LM and Encoder only).

Script	**Method**	**Key outputs (Simulation_Results/)**
LM_analysis_Simulation_data_iterspecific_smoothed.py	Levenberg–Marquardt, per-iteration threshold calibration + smoothing	LM_metrics_vs_iteration.csv/.png, LM_predictions_long.csv, LM_confusion_matrix_last_iter.png, LM_correctness_heatmap.png
Bayesian_analysis_simulation_data_iterspecific.py	Sequential Bayesian inference, per-chain-position likelihood calibration	Bayesian_metrics_vs_iteration.csv/.png, Bayesian_predictions_long.csv, Bayesian_confusion_matrix_last_iter.png, Bayesian_correctness_heatmap.png
Encoder_analysis_simulation_data_smoothed.py	Feedforward neural network (scikit-learn MLPClassifier) + smoothing	Encoder_metrics_vs_iteration.csv/.png, Encoder_predictions_long.csv, Encoder_confusion_matrix_last_iter.png, Encoder_correctness_heatmap.png

Each can be run independently and in any order, once Simulation_Results/ exists:

python LM_analysis_Simulation_data_iterspecific_smoothed.py

python Bayesian_analysis_simulation_data_iterspecific.py

python Encoder_analysis_simulation_data_smoothed.py

Runtime note: the LM script recalibrates its decision threshold separately at every evaluated iteration, which costs substantially more compute than a single fixed-threshold calibration — expect it to be the slowest of the three on a full 5000-file run.

Split-stability analysis and summary figure (optional)
These six scripts are not part of the main pipeline. They use the outputs of the pipeline above to check how much the reported results depend on one particular random train/test split, and to present the headline comparison as a figure. All six expect to sit in the same folder as the three classifier scripts, with Simulation_Results/ inside it, and need no arguments.

run_seed_sweep.py
Reads: Simulation_Results/ and the three classifier scripts above. **Writes**: seed_sweep_results/seed_<N>/{LM,Bayesian,MLP}_predictions_long.csv

Repeats the full 5-fold stratified cross-validation of all three classifiers under several different random seeds, i.e. several different assignments of files to folds. The classifier scripts themselves are not modified: each one takes all of its randomness from SETTINGS["random_state"] (the fold assignment for all three; for the MLP also the oversampling and network initialisation), so this script imports each one, overrides that single setting, runs the script's own cross-validation function, applies the script's own smoothing (LM and MLP only, as in the original scripts), and writes predictions in the same format as the original *_predictions_long.csv files.

The default seeds are 42 and 101–109 (edit DEFAULT_SEEDS near the top of the script to change them); each method for each seed is written only when it has fully finished, and finished runs are skipped, so an interrupted run can simply be restarted. Seed 42 is the seed used for the main analysis, so its output should reproduce the main results exactly. Options (all optional): --seeds, --methods {LM,Bayesian,MLP}, --scripts-dir, --sim-dir, --out-dir.

Runtime note: this repeats the full classifier runs once per seed, so it is expensive. On a 5000-file dataset expect on the order of hours per seed, with the LM classifier the slowest (it recalibrates a threshold at every evaluated iteration). Run a single seed first (python run_seed_sweep.py --seeds 42) to gauge the time on your hardware.

Run: python run_seed_sweep.py

summarize_seed_sweep.py

Reads: seed_sweep_results/ (run_seed_sweep.py's output; only seeds for which all three methods have finished are used) **Writes**: into seed_sweep_results/: seed_sweep_per_seed.csv, seed_sweep_summary.csv, seed_sweep_rankings.csv, seed_sweep_figure.png

For each seed it recomputes Recall, Precision and Leakage at iterations 10 and 300 from the pooled out-of-fold predictions (as in the paper's Table 1), and the majority-vote ensemble at iteration 300 (see summarize_method_selection.py and SI Section S7). It then reports, across seeds, the mean, standard deviation, minimum and maximum of each metric, and for how many seeds each method ranking discussed in the paper holds (for example "MLP has the lowest leakage at iteration 10"). The figure shows the per-seed values with their mean. It can be run at any point during a sweep, on whichever seeds are complete. With few seeds, the min–max range and the count of seeds in which a ranking holds are more meaningful than the standard deviation. The convergence and overlap analyses (Tables 2 and 3) are handled by summarize_seed_convergence_overlap.py.

Run: python summarize_seed_sweep.py

summarize_seed_convergence_overlap.py
Reads: seed_sweep_results/ (run_seed_sweep.py's output; only seeds for which all three methods have finished are used) **Writes**: into seed_sweep_results/: seed_sweep_convergence.csv, seed_sweep_overlap.csv, seed_sweep_vote_counts.csv, seed_sweep_persistence.csv

For each seed it recomputes, from the per-iteration predictions of the three classifiers, (i) the convergence speed and smoothness of the precision-versus-iteration curve (as in the paper's Table 2): the first evaluated iteration at which precision reaches 0.99 and the number of reversals (direction changes between consecutive non-zero steps of the curve evaluated at iterations 10, 20, ..., 300); (ii) the overlap in misclassified true-single datasets at iteration 300 (as in Table 3 and Figure S1): total, unique and shared misses per method, and the number of datasets missed by all three, exactly two and exactly one method (a true single is missed by the majority-vote ensemble if and only if at least two methods miss it); and (iii) how persistent the misses are across seeds, i.e. how many distinct datasets each method misses in at least one seed and in every seed. The three methods read different file variants of the same simulated dataset (raw or normalised counts), so datasets are matched across methods by their repetition identifier (rep_<i>_n<N>). It can be run at any point during a sweep, on whichever seeds are complete.

Run: python summarize_seed_convergence_overlap.py

summarize_time_to_operating_point.py

Reads: seed_sweep_results/ (run_seed_sweep.py's output; only seeds for which all three methods have finished are used) and, optionally, iter_to_time_mapping_lookup.csv (written by Compute_iter_to_integration_time_mapping.py) **Writes**: into seed_sweep_results/: seed_sweep_time_to_operating_point.csv,
seed_sweep_time_to_operating_point_summary.csv

Expresses the convergence speed of the three classifiers as the acquisition time each one needs to reach a given operating point (the paper's Table 6), instead of as an abstract iteration number. For every seed and method it finds the first evaluated iteration from which a criterion is met and stays met at all later evaluated iterations (iterations 10, 20, ..., 300; pooled out-of-fold predictions). The criteria are leakage ≤ 1%, leakage ≤ 0.5%, precision ≥ 0.99, and precision ≥ 0.99 together with recall ≥ 0.95. If the lookup of simulated iteration to equivalent real integration time is available (by default iter_to_time_mapping_lookup.csv next to the script, or pass --mapping-csv), the iterations are also converted to seconds; otherwise only iterations are reported. The mapping is representative of the emitters and detectors used to build it, not universal: the baseline count per histogram bin scales with the product of the two detector count rates, so a dimmer emitter needs proportionally more real time for the same iteration.

Run: python summarize_time_to_operating_point.py

summarize_method_selection.py
Reads: seed_sweep_results/ (run_seed_sweep.py's output; only seeds for which all three methods have finished are used) and, optionally, iter_to_time_mapping_lookup.csv Writes: into seed_sweep_results/: method_selection_table.csv (every seed, method and iteration), method_selection_summary.csv (primary seed with 95% Wilson intervals, plus mean and range over seeds) and ensemble_vs_bayesian_paired.csv

Quantitative basis for choosing a classifier as a function of the acquisition time (the paper's Table 5 and SI Table S7), and paired comparison of the majority-vote ensemble with the Bayesian classifier alone (SI Section S7, Table S8). For each seed it computes recall, precision and leakage of LM, Bayesian, MLP and of the majority vote at a set of iterations (default 10, 30, 50, 60, 100, 200 and 300; change with --iterations), with 95% Wilson score intervals and, when the mapping lookup is available, the equivalent real time. The majority vote classifies a dataset as multi if at least two of the three methods do so. The paired comparison counts, per seed, the true singles rescued by the vote (missed by the Bayesian classifier, caught by the vote) and lost by it, gives the exact two-sided McNemar (binomial) p-value on these discordant datasets, and the false positives of the Bayesian classifier and

of the vote. The ten seeds re-use the same 5000 simulated datasets, so the per-seed p-values are not independent and are not combined.

Run: python summarize_method_selection.py

make_table1_barplot.py
Reads: LM_predictions_long.csv, Bayesian_predictions_long.csv, Encoder_predictions_long.csv (written by the three classifier scripts; by default looked up in the current directory, or pass --data-dir) **Writes**: Figure_Table1_barplot.png, Figure_Table1_barplot_caption.txt, table1_with_CI.csv

Draws a two-panel bar plot of Recall, Precision and Leakage for the three classifiers at the sparsest (iteration 10) and most-converged (iteration 300) integration times, with 95% Wilson score confidence intervals as error bars. The Wilson interval is used rather than the normal approximation because it remains well defined when a proportion is exactly 0 or 1, which happens routinely for Leakage; the intervals are therefore not symmetric about the point estimate near 0 or 1. At iteration 300 the Leakage values are shown in a small log-scale inset, using the same zero-value floor convention as the Leakage panel of the main metrics-vs-iteration figure. table1_with_CI.csv contains the plotted values together with their confidence bounds and the underlying TP/FN/FP/TN counts. Use --no-leakage-log-inset to disable the inset.

Run: python make_table1_barplot.py

Supporting analyses (optional)
These scripts are not part of the main pipeline. They provide the iteration-to-time correspondence, a control analysis with a fixed lifetime, the computational-cost measurement and the characterization of the experimental emitters. Unless stated otherwise, they expect to sit in the same folder as the three classifier scripts, with Simulation_Results/ and Experimental_Data_Raw/ inside it.

Compute_iter_to_integration_time_mapping.py
Reads: Experimental_Data_Raw/ and Simulation_Results/ (Step_02's output). **Writes**: iter_to_time_mapping_lookup.csv, iter_to_time_mapping_detailed.csv, iter_to_time_mapping.png.

Translates the simulated iteration number into an approximate equivalent real integration time. The iteration number has no time unit of its own, and different emitters accumulate counts at different rates, so the two axes are matched through the baseline count rate (mean number of coincidences per bin in the flat far-delay region), which can be computed on

equal footing for real and simulated data: for the real files from the accidental-coincidence formula $r_1 \cdot r_2 \cdot \Delta\tau \cdot T$ at each of the nine known integration times, and for the simulated files from the far-tail mean of the raw counts at each iteration. Simulated iterations are mapped to real times by log–log interpolation of the real curve, for every real file individually and as a pooled median with its interquartile range; iterations whose baseline lies outside the range covered by the real data are extrapolated and flagged in the output. The result is a representative correspondence, not a universal one: a dimmer emitter needs proportionally longer real acquisition to reach the same baseline. For the data of the paper, iteration 300 corresponds to about 79 s. The lookup table is provided in the repository, so that the scripts that use it (summarize_time_to_operating_point.py, summarize_method_selection.py, Figure_real_vs_simulated_with_mapping.py) can be run without repeating this step. If your acquisition used a different schedule, edit real_integration_times_s in SETTINGS.

Run: python Compute_iter_to_integration_time_mapping.py

Figure_real_vs_simulated_with_mapping.py
Reads: one corrected real file from Experimental_Data/ (Step_00's output), one simulated N=1 file from Simulation_Results/, and iter_to_time_mapping_lookup.csv. **Writes**: figure_real_vs_simulated_with_mapping.png

Two-part figure. Left: a real N=1 $g^2(\tau)$ trace (top row) and a simulated N=1 trace whose injected background fraction is close to the calibrated value of the real file (bottom row), at three matched integration times (by default 2, 10 and 50 s, i.e. real iterations 2, 4 and 6). The matching simulated iterations are looked up from the mapping curve rather than chosen by hand, so the two rows correspond to the same integration times; the real times must lie within the mapped range (up to about 79 s). Right: the iteration-to-integration-time correspondence curve with the three simulated iterations marked. The files to plot, the real iterations to show and the output name are set in SETTINGS.

Run: python Figure_real_vs_simulated_with_mapping.py

LM_analysis_fixed_tau1.py
Reads: Simulation_Results/ and the standard LM and Bayesian scripts, which are imported (same folder). **Writes**: LM_fixed_tau1_*.csv (metrics versus iteration, smoothed and unsmoothed; out-of-fold predictions for each variant; optionally the same for the Bayesian reference).

Levenberg–Marquardt classification with the antibunching lifetime τ_1 fixed beforehand, run on exactly the same files, 5-fold stratified split (seed 42), tail normalization, Poisson weights, per-iteration midpoint threshold and EMA smoothing as the standard LM script. Three variants are compared fold by fold: LM_free (all parameters free, as in the standard LM

script), LM_tau1_A1 (τ_1 fixed to the calibrated value of the fold, the amplitude A_1 being the only fit parameter) and LM_tau1_A1_A2 (τ_1 fixed; amplitude A_1 and pedestal A_2 free). τ_1 is calibrated exactly as in the Bayesian script: the median τ_1 of the full fit to the last iteration of the training files of each fold. With τ_1 fixed the model is linear in the amplitudes, so each fit is a closed-form weighted least-squares solution. Optionally (run_bayesian_reference in SETTINGS) the Bayesian classifier is run on the same folds for reference. See SI Section S8.

Run: python LM_analysis_fixed_tau1.py

Benchmark_computational_cost.py

Reads: Simulation_Results/ (both the g2_raw_* and the g2_norm_* files) and the three classifier scripts, which are imported unchanged (same folder). **Writes**: Benchmark_computational_cost.csv, Benchmark_computational_cost_environment.txt

Wall-clock cost of the three classifiers on one fold of the 5-fold split (seed 42), single-threaded and in memory (the files are read once before the timed sections). It reports (1) the one-off cost on the training datasets: calibration of the per-iteration thresholds (LM), calibration of τ_1 and of the likelihoods (Bayesian), and feature construction and training (MLP); and (2) the cost of classifying a new dataset once calibrated or trained: the whole 30-point trajectory (iterations 10, 20, …, 300) and one additional histogram, i.e. one more update during an acquisition. The LM and Bayesian calibration costs are strictly proportional to the number of training datasets, so they are timed on a random subset (n_calib_subset, default 240) and extrapolated linearly to the full training set of a fold (4,000); the MLP is trained on the complete training set. The absolute times depend on the hardware; the ratios between the methods are more transferable. See SI Section S9 and Table S10.

Run: python Benchmark_computational_cost.py

fit_pedestal_A2.py

Reads: Experimental_Data/ (Step_00's output: the normalized files and their _rawcounts companions). Run from the folder that contains Experimental_Data/. **Writes**: A2_fits.csv

Fits the reduced model of Eq. (1), $g^2(\tau) = 1 - A_1\exp(-|\tau|/\tau_1) + A_2$, to the corrected experimental histograms at the three longest integration times (100, 200 and 500 s), by weighted nonlinear least squares with Poisson weights, and reports for each file the pedestal A_2 (with its fit uncertainty), τ_1, the zero-delay value normalized to the long-time (accidental) level, $g^2(0) = 1 - A_1 + A_2$, and the value normalized to the local plateau of the pedestal, $g^2(0)/(1 + A_2)$. At the longest integration time it prints the median and interquartile range of A_2 for each emitter number N. For reference, for the 13 N=1 files the median is 0.34 (interquartile range 0.29–0.46). See SI Section S10.2.

Run: python fit_pedestal_A2.py

Figure_S_emitter_count_rates.py

Reads: Experimental_Data_Raw/ (the original raw files, with the APD1/APD2 rate rows; a different folder can be passed as the first argument). **Writes**: Figure_S_emitter_count_rates.png, Emitter_count_statistics.csv, Emitter_count_statistics_summary.csv

Detected count rates, stability of the rate and coincidence levels of the 25 experimental emitters. For each emitter it computes the total detected rate (APD1 + APD2) at each integration time and at 500 s, the relative standard deviation (RSD) of the total rate over the nine integration times, and the mean number of coincidences per 0.1-ns bin at far delays ($|\tau| > 80$ ns, after re-centering the antibunching dip at zero delay with the fixed timing offset of 18.1 ns), together with the ratio of the far-delay level to the accidental level $r_1 \cdot r_2 \cdot \Delta\tau \cdot T$ at 500 s. The figure shows (a) the total rate at 500 s, (b) the total rate of one representative emitter at the nine integration times, (c) the RSD of each emitter and (d) the coincidences per bin as a function of integration time. For reference, the total rate at 500 s ranges from 67 to 642 kcps (median 142 kcps) and the RSD is 1.6% (median) and below 9% for all emitters. Set SHOW_PANEL_LETTERS to draw the panel letters on the figure. See SI Section S10.2 (Figure S4 and Table S11).

Run: python Figure_S_emitter_count_rates.py

Reproducibility notes

- All three classifiers use random_state=42 for their cross-validation split; results should be exactly reproducible given the same Simulation_Results/ data.
- Changing random_state changes which files land in which fold, and (for the MLP) the oversampling and network initialisation. run_seed_sweep.py automates repeating the analysis with other seeds; its seed-42 run should match the main results exactly. The simulated dataset itself is held fixed across seeds. summarize_seed_sweep.py and summarize_seed_convergence_overlap.py summarize the results across seeds.
- Step_02's Monte Carlo generation is *not* seeded, so re-running it produces a different (though statistically similar) synthetic dataset each time. If you need a byte-identical dataset across runs, add an explicit np.random.seed(…) near the top of Step_02 before generation begins.

- Every script prints a full log of what it did (files excluded, folds calibrated, parameters used) to the console — worth capturing (python Step_00_....py | tee step00_log.txt) alongside your results for later reference.
- The analyses of the experimental data (fit_pedestal_A2.py, Figure_S_emitter_count_rates.py) are deterministic. The timings written by Benchmark_computational_cost.py depend on the hardware and on the software versions, which are recorded in Benchmark_computational_cost_environment.txt.